

IMPLEMENT CLIENT SCRIPT & UI POLICY (INCIDENT)

Problem Statement

Incident records often require consistent and accurate data entry to ensure effective triage, routing, and resolution. However, relying solely on user awareness and manual checks can lead to incomplete, inconsistent, or incorrect data being submitted.

This can impact reporting accuracy, SLA compliance, and overall service quality. To address this challenge, there is a need to enforce conditional field behavior and validation directly at the user interface level.

Implementing standardized UI Policies and Client Scripts helps control field visibility, mandatory status, and save-time validation, ensuring that Incident records meet defined business rules before they are submitted.

Objective

The objective of the Implement Client Script & UI Policy (Incident) project is to demonstrate how ServiceNow client-side controls can be used to enforce data integrity on Incident records.

This implementation showcases how UI Policies and Client Scripts can dynamically make fields mandatory, auto-populate values, control field behavior, and prevent record submission when required conditions are not met.

The solution ensures that Incidents are created with complete and valid information, improving data quality and consistency across the platform.

Skills

- Incident Management
- UI Policy
- UI Policy Actions
- Client Scripts
- Form Validation
- ServiceNow

Task 1: Create UI Policy on Incident

Activity 1: Create UI Policy

1. Log in to the ServiceNow instance with administrative or configuration access.
2. Navigate to All → System UI → UI Policies.
3. Click New.
4. Select Incident [incident] as the table.
5. Enter the required UI Policy details.
6. Define the required condition based on the business requirement.

7. Configure the required UI Policy Actions.

8. Click Submit or Update to save the UI Policy.

Expected Result

The UI Policy should dynamically control the Incident form based on the specified condition.

Task 2: Configure UI Policy Actions

Activity 1: Create UI Policy Action

1. Open the UI Policy created for the Incident table.

2. Go to the UI Policy Actions section.

3. Click New.

4. Select the required Incident field.

5. Configure the field properties such as:

- Mandatory

- Visible

- Disabled

6. Set the required values according to the business requirement.

7. Save the UI Policy Action.

Expected Result

The selected Incident fields should automatically change their behavior when the UI Policy condition is satisfied.

Task 3: Create Client Script

Activity 1: Create Client Script

1. Log in to the ServiceNow instance.

2. Navigate to All → System Definition → Client Scripts.

3. Click New.

4. Select Incident [incident] as the table.

5. Select the appropriate Client Script type.

6. Enter the required field and condition.

7. Write the required JavaScript code according to the business requirement.

8. Save the Client Script.

Purpose

Client Scripts are used to control and validate Incident form behavior on the client side.

Expected Result

The Client Script should execute the defined action when the specified condition occurs.

Task 4: Configure Client Script

Activity 1: Define Client Script Behavior

The Client Script can be configured to perform actions such as:

- Making fields mandatory
- Setting field values automatically
- Showing or hiding fields
- Enabling or disabling fields
- Validating user input
- Displaying messages to users

The script should be written according to the required Incident management business rule.

Expected Result

The Incident form should respond automatically when the specified field value or form action occurs.

Task 5: Form Validation

Activity 1: Validate Incident Data

1. Open an Incident record.
2. Enter the required information.
3. Change the field value that triggers the UI Policy or Client Script.
4. Check whether the required fields are automatically controlled.
5. Try to submit the Incident without entering required information.
6. Verify that the Client Script prevents incorrect or incomplete submission when required.

Expected Result

The system should prevent invalid or incomplete Incident records from being submitted.

Task 6: Testing

Activity 1: Test UI Policy

1. Open a new Incident record.
2. Enter the required field value.
3. Check the UI Policy behavior.
4. Verify the field visibility, mandatory status, and disabled status.

Activity 2: Test Client Script

1. Open an Incident record.
2. Perform the action that triggers the Client Script.
3. Verify the expected field behavior.
4. Check whether the validation message is displayed when required.

Expected Result

Both UI Policy and Client Script should work according to the defined business requirements.

Task 7: Expected Result

After implementing the UI Policy and Client Script:

- Required fields are controlled automatically.
- Field visibility is managed dynamically.
- Fields can be enabled or disabled based on conditions.
- Invalid data can be prevented from being submitted.
- Incident records contain more complete and accurate information.
- Data consistency is improved.
- User interface behavior becomes standardized.

Task 8: Final Result

The Client Script & UI Policy implementation for Incident successfully demonstrates client-side control and validation in ServiceNow.

The solution helps ensure that Incident records follow the required business rules before submission. UI Policies control field behavior dynamically, while Client Scripts provide validation and automated actions.

This improves the accuracy, consistency, and quality of Incident data.

Screenshots

The following screenshots can be added as evidence of the implementation:

1. ServiceNow Login / Instance
2. Incident Form
3. UI Policy Creation
4. UI Policy Conditions
5. UI Policy Actions
6. Client Script Creation
7. Client Script Configuration
8. Form Validation
9. Testing
10. Final Result

Conclusion

The Implement Client Script & UI Policy (Incident) project demonstrates how ServiceNow client-side features can be used to improve Incident data quality.

UI Policies provide dynamic control over fields, while Client Scripts provide validation and automation. Together, they help maintain consistent and accurate Incident records and support better Incident management.